

M05 / ASSESSMENT

M05 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
Rules	3
T01 - DE05-01 CSV properly: parse, inspect, preserve raw	3
T02 - DE05-02 JSON and JSONL from zero	3
T03 - DE05-03 Paths, folders and deterministic batch files	3
T04 - DE05-04 pandas profiling before transformation	3
T05 - DE05-05 Cleaning with pandas and rejected rows	4
T06 - DE05-06 pandas merge, cardinality and aggregation	4
T07 - DE05-07 Data validation: record, batch and relationship rules	4
T08 - DE05-08 HTTP and REST APIs from zero	4
T09 - DE05-09 Calling APIs safely with requests	4
T10 - DE05-10 Pagination and authentication without leaking secrets	4
T11 - DE05-11 Retries, rate limits and partial failure	4
T12 - DE05-12 Python + PostgreSQL with SQLAlchemy	5
T13 - DE05-13 Configuration, environment variables and secret hygiene	5
T14 - DE05-14 Logging, testing and reusable pipeline code	5
T15 - DE05-15 Integrated Python data-engineering pipeline	5

Read this first

MODULE REVISION TEST

Save your answers before opening the protected answer guide. No forced timer. First prove the competency untimed; later you may record interview-style time if useful. This test does not replace a protected Gate.

Coverage: 15 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

Rules

Work with learner book, review book, answer guide and AI solving closed. Save one answer/evidence item for every lesson. If a task needs a PC, write your prediction first, then run it. Mark PASS only when the result and your explanation agree.

T01 - DE05-01 | CSV properly: parse, inspect, preserve raw

Create a new CSV with a quoted city value Delhi, NCR and an ID 0009. Parse it without losing field boundaries or leading-zero identity.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T02 - DE05-02 | JSON and JSONL from zero

Create a three-line JSONL file where the second event has an optional nested metadata object. Process each line and explain compatibility without forcing all events to identical shape.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T03 - DE05-03 | Paths, folders and deterministic batch files

Use filenames whose lexical order conflicts with a business_date inside each file. Implement and explain the intended ordering rule.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T04 - DE05-04 | pandas profiling before transformation

Profile a changed file containing a blank numeric field and one duplicate key; do not clean it. Produce only observations and suspected risks.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T05 - DE05-05 | Cleaning with pandas and rejected rows

Use the fresh inventory fixture: valid unique SKU, positive integer quantity, nonnegative decimal cost and ISO received date. Split accepted/rejected with reasons.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T06 - DE05-06 | pandas merge, cardinality and aggregation

Create a small product dimension with one duplicate key and prove a many-to-one merge guard fails before aggregation; then repair the dimension and reconcile.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T07 - DE05-07 | Data validation: record, batch and relationship rules

Add a syntactically valid customer_id that does not exist in the dimension. Show record validation passes while relationship validation fails.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T08 - DE05-08 | HTTP and REST APIs from zero

Add a local endpoint/fixture expectation where status is 200 but required field is absent. Explain why transport succeeded while contract validation failed.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T09 - DE05-09 | Calling APIs safely with requests

Call a path returning 404 and prove your client does not misclassify it as JSON success or transient server failure.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T10 - DE05-10 | Pagination and authentication without leaking secrets

Fetch the same seven records with a different page size and order of page requests under a controlled replay; prove logical completeness/uniqueness remains the same.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T11 - DE05-11 | Retries, rate limits and partial failure

Change retry budget from 4 to 2 and explain what evidence would show final failure. Do not create an infinite retry loop.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T12 - DE05-12 | Python + PostgreSQL with SQLAlchemy

Insert a different two-row dataset inside one transaction, deliberately violate a unique key, and prove the transaction boundary behavior.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T13 - DE05-13 | Configuration, environment variables and secret hygiene

Switch the input directory via environment variable without editing program logic; then remove one required variable and prove failure occurs before outputs.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T14 - DE05-14 | Logging, testing and reusable pipeline code

Write a test for invalid date and one for duplicate ID policy. Break one rule intentionally, observe failure, repair only that rule and rerun.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T15 - DE05-15 | Integrated Python data-engineering pipeline

After reviewing your weak points, close all review material and solve M05_Fresh_Checkpoint_TODO.py from the fresh inventory fixture. Keep it separate from the first attempt.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY